UNIVERSIDADE POLITÉCNICA DO CÁVADO E DO AVE

ESCOLA SUPERIOR DE TECNOLOGIA UPCA

Pipeline ETL para Análise do Poder de Compra no Setor Tecnológico

Tech Purchasing Power

Trabalho Prático I

Integração de Sistemas de Informação

Alexandre Silva

Nº 29382

Orientação

Luís Ferreira

Ano letivo 2026/2027

Licenciatura em Engenharia de Sistemas Informáticos

Escola Superior de Tecnologia

Universidade Politécnica do Cávado e do Ave

Identificação do Grupo

O presente trabalho foi realizado a título individual, pelo elemento identificado na tabela seguinte, no âmbito da unidade curricular de Integração de Sistemas de Informação (ISI).

Elemento	Número de aluno
Alexandre Silva	29382

Orientação

Luís Ferreira

Docente da unidade curricular de Integração de Sistemas de Informação

Informação sobre o Trabalho

Campo	Detalhe
Tema	Pipeline ETL para Análise do Poder de Compra no Setor Tecnológico (*Tech Purchasing Power*)
Curso	Licenciatura em Engenharia de Sistemas Informáticos
Ano letivo	2026/2027
Email	asilvaofficial@gmail.com
Plataformas utilizadas	KNIME Analytics Platform, Node-RED, RabbitMQ, PostgreSQL, Grafana e Docker
Repositório Git	https://github.com/Alexms22/tech-purchasing-power

RESUMO

O presente relatório documenta o Trabalho Prático I da unidade curricular de Integração de Sistemas de Informação, dedicado à conceção e implementação de um pipeline ETL para a análise do poder de compra no setor tecnológico (Tech Purchasing Power). O problema abordado prende-se com a dificuldade de comparar ofertas de emprego tecnológicas a nível mundial, uma vez que os salários surgem em moedas e periodicidades distintas, as localizações são inconsistentes e o salário bruto não traduz o poder de compra real depois de descontados os impostos, a renda e o custo de vida.

Para o resolver, desenvolveu-se uma arquitetura híbrida com duas vias complementares. A via em lote, implementada no KNIME Analytics Platform, consome três formatos distintos — JSON de APIs REST, CSV de um ficheiro histórico e HTML extraído por XPath da página de índices da Numbeo —, concatena e padroniza as ofertas, cruza-as com os indicadores económicos de cada país e calcula a Poupança Anual Estimada. A via contínua, implementada em Node-RED com RabbitMQ, recolhe novas ofertas de hora a hora, normaliza-as e persiste-as por escrita por cima, despachando ainda alertas automáticos via Telegram. Ambas convergem para a mesma base de dados PostgreSQL, executada em Docker, que alimenta um dashboard interativo no Grafana com filtros dinâmicos por país e nível de experiência.

Os resultados mostram que o salário bruto mais elevado nem sempre corresponde à maior capacidade de poupança: os Estados Unidos e o Canadá lideram a poupança anual média estimada, acima dos 60 mil euros, enquanto Singapura, apesar dos salários elevados, apresenta valores substancialmente inferiores. A solução demonstra a eficácia de combinar processamento analítico em lote com ingestão contínua desacoplada na integração de dados heterogéneos.

Palavras-chave: ETL; Integração de Sistemas de Informação; KNIME; Node-RED; RabbitMQ; PostgreSQL; Grafana; Docker; poder de compra.

ABSTRACT

This report documents Practical Assignment I of the Information Systems Integration course, which focuses on the design and implementation of an ETL pipeline for analysing purchasing power in the technology sector (Tech Purchasing Power). The problem addressed is the difficulty of comparing technology job offers worldwide: salaries are expressed in different currencies and time periods, locations are inconsistent, and gross salary does not reflect real purchasing power once taxes, rent and cost of living are taken into account.

To solve it, a hybrid architecture with two complementary paths was developed. The batch path, built in KNIME Analytics Platform, consumes three distinct formats — JSON from REST APIs, CSV from a historical file, and HTML scraped with XPath from the Numbeo index page —, concatenates and standardises the offers, joins them with each country economic indicators and computes an Estimated Annual Savings figure. The streaming path, built in Node-RED with RabbitMQ, collects new offers hourly, normalises them and stores them through an upsert, while dispatching automatic Telegram alerts. Both converge on the same PostgreSQL database, running in Docker, which feeds an interactive Grafana dashboard with dynamic filters by country and experience level.

The results show that the highest gross salary does not always correspond to the greatest saving capacity: the United States and Canada lead the estimated average annual savings, above 60 thousand euros, whereas Singapore, despite high salaries, shows substantially lower values. The solution demonstrates the effectiveness of combining batch analytical processing with decoupled continuous ingestion when integrating heterogeneous data.

Keywords: ETL; Information Systems Integration; KNIME; Node-RED; RabbitMQ; PostgreSQL; Grafana; Docker; purchasing power.

ÍNDICE

1. Introdução e Enquadramento.....1
1.1. Enquadramento e Contexto.....1
1.2. Descrição do Problema e sua Relevância.....2
1.3. Motivação e Objetivos.....2
1.4. Metodologia e Estrutura do Documento.....3
2. Estratégia Utilizada e Arquitetura.....5
2.1. Arquitetura Híbrida: Processamento em Lote e Contínuo.....5
2.2. Ambiente Contentorizado (Docker).....7
2.3. Justificação das Escolhas Tecnológicas.....8
2.4. Evolução da Solução e Abordagens Exploradas.....9
2.4.1. Tentativa de Diversificação com MongoDB.....10
2.4.2. Justificação da Opção Final pelo PostgreSQL.....10
2.5. Cobertura das Fases do ETL.....11
2.5.1. Fase 1 – Recolha e Preparação de Dados (E).....11
2.5.2. Fase 2 – Processamento e Transformação de Dados (T).....11
2.5.3. Fase 3 – Persistência e Exportação (L).....11
3. Transformações.....13
3.1. Via em Lote: Workflow no KNIME Analytics Platform.....13
3.1.1. Ramo 1 – Extração via API REST (JSON).....14
3.1.2. Ramo 2 – Extração em Lote de Ficheiro Local (CSV).....15
3.1.3. Ramo 3 – Extração Web com XPath (HTML).....15
3.1.4. Fusão e Padronização Global.....16
3.1.5. Cruzamento, Modelação Fiscal e Cálculo da Poupança.....17
3.2. Via Contínua: Transformações no Node-RED.....18
3.2.1. Preparação dos Termos e Chamadas às APIs.....18
3.2.2. Regras de Negócio no Nó Normal Data.....19
3.2.3. Reconstrução e Validação de Parâmetros (Rebuild Params).....20
4. Jobs e Persistência de Dados.....21
4.1. Job de Execução Cíclica no KNIME.....21
4.2. Job de Agendamento e Mensageria Assíncrona (RabbitMQ).....21
4.3. Job de Carregamento e Escrita por Cima (Upsert).....23
4.4. Modelo de Dados no PostgreSQL.....23
4.4.1. Tabela ofertas_emprego (via em lote).....24
4.4.2. Tabela vagas_nodered (via contínua).....24
4.5. Job de Notificação: Alertas em Tempo Real via Telegram.....25
4.6. Diagnóstico Operacional e Limitações.....27
5. Visualização Analítica e Discussão de Resultados.....29
5.1. Ligação e Variáveis Dinâmicas no Grafana.....29
5.2. Processo de Configuração de um Painel.....29
5.3. Análise dos Painéis do Dashboard Tech Power Purchasing.....30
5.3.1. Top 10 Países com Maior Poupança Anual.....31
5.3.2. Poder de Compra – Média Salarial Anual.....32

5.3.3. Salários por País e Senioridade.....32
5.3.4. Distribuição de Vagas por País.....32
5.4. Síntese da Discussão e Limitações.....33
6. Vídeo com Demonstração.....35
7. Conclusão e Trabalhos Futuros.....36
7.1. Conclusão.....36
7.2. Trabalhos Futuros.....36
8. Referências Bibliográficas.....38

ÍNDICE DE FIGURAS

Figura 1: Arquitetura híbrida do pipeline ETL Tech Purchasing Power.....6
Figura 2: Contentores do pipeline em execução no Docker Desktop.....7
Figura 3: Workflow KNIME (1 de 2): ramos de extração e preparação.....13
Figura 4: Workflow KNIME (2 de 2): fusão, enriquecimento e persistência.....13
Figura 5: Ramo de extração via API REST e resolução de códigos postais (KNIME).....14
Figura 6: Ramo de extração e normalização do ficheiro CSV (KNIME).....15
Figura 7: Ramo de extração dos índices económicos do Numbeo por XPath (KNIME).....16
Figura 8: Concatenação e regras de padronização global (KNIME).....16
Figura 9: Junção com os índices Numbeo, cálculo da poupança e escrita em base de dados (KNIME).....17
Figura 10: Fluxo produtor de extração e transformação no Node-RED.....18
Figura 11: Nós de controlo do ciclo de execução periódica no KNIME.....21
Figura 12: Painel de gestão do RabbitMQ (separador Overview).....22
Figura 13: Fluxo consumidor e carregamento no PostgreSQL (Node-RED).....23
Figura 14: Registos da tabela ofertas_emprego, escrita pelo KNIME (200 021 registos).....25
Figura 15: Registos da tabela vagas_nodered, escrita pelo Node-RED (309 registos).....25
Figura 16: Notificações recebidas no bot @TechPowerISIBot do Telegram.....27
Figura 17: Configuração da visualização de um painel no Grafana.....30
Figura 18: Editor de consultas SQL de um painel no Grafana.....30
Figura 19: Dashboard Tech Power Purchasing no Grafana.....31
Figura 20: Código QR com a ligação para o vídeo de demonstração do pipeline.....35

ÍNDICE DE TABELAS

Tabela 1: Correspondência entre as fases do ETL, os componentes e os capítulos do documento..... 4
Tabela 2: Correspondência entre os pontos exigidos no enunciado e a sua localização neste documento..... 4
Tabela 3: Comparação entre as duas vias de processamento do pipeline..... 7
Tabela 4: Contentores que compõem o ambiente de execução..... 8
Tabela 5: Abordagens exploradas ao longo do desenvolvimento e decisão final..... 9
Tabela 6: Formatos e origens de dados consumidos pelo pipeline..... 11
Tabela 7: Indicadores globais observados no painel de gestão do RabbitMQ..... 22
Tabela 8: Colunas da tabela ofertas_emprego, escrita pelo KNIME..... 24
Tabela 9: Critérios de disparo de alertas no nó Good Role Alert..... 26
Tabela 10: Número de vagas por mercado no painel Distribuição de Vagas por País..... 33

Siglas e Acrónimos

AMQP – Advanced Message Queuing Protocol (Protocolo Avançado de Filas de Mensagens)

API – Application Programming Interface (Interface de Programação de Aplicações)

BD – Base de Dados

CSV – Comma-Separated Values (Valores Separados por Vírgulas)

DOM – Document Object Model (Modelo de Objetos do Documento)

ER – Expressão Regular (*Regex*)

ETL – Extract, Transform and Load (Extração, Transformação e Carregamento)

HTML – HyperText Markup Language (Linguagem de Marcação de Hipertexto)

HTTP – HyperText Transfer Protocol (Protocolo de Transferência de Hipertexto)

ISI – Integração de Sistemas de Informação

JDBC – Java Database Connectivity

JSON – JavaScript Object Notation

NLP – Natural Language Processing (Processamento de Linguagem Natural)

NoSQL – Not only SQL

QR – Quick Response

REST – Representational State Transfer

SQL – Structured Query Language (Linguagem de Consulta Estruturada)

TP1 – Trabalho Prático I

UC – Unidade Curricular

UPCA – Universidade Politécnica do Cávado e do Ave

URL – Uniform Resource Locator (Localizador Uniforme de Recursos)

XPath – XML Path Language (Linguagem de Caminhos para XML)

1. Introdução e Enquadramento

O presente documento constitui o relatório final do Trabalho Prático I (TP1) da unidade curricular (UC) de Integração de Sistemas de Informação (ISI). O trabalho consistiu na conceção, implementação e demonstração de um *pipeline* de ETL (*Extract, Transform and Load*) dedicado à análise do poder de compra de profissionais do setor tecnológico, designado ao longo do documento por *Tech Purchasing Power*. Neste capítulo apresentam-se o enquadramento do trabalho, o problema que se pretende resolver, a motivação e os objetivos, bem como a metodologia seguida e a estrutura do documento.

1.1. Enquadramento e Contexto

O trabalho foi desenvolvido no âmbito da UC de ISI, integrada na Licenciatura em Engenharia de Sistemas Informáticos da Escola Superior de Tecnologia da Universidade Politécnica do Cávado e do Ave (UPCA), no ano letivo 2026/2027, sob orientação do docente Luís Ferreira. Foi realizado individualmente por Alexandre Silva (n.º 29382), tendo sido exploradas, de forma integrada, duas plataformas de ETL complementares — o KNIME Analytics Platform e o Node-RED — apoiadas em RabbitMQ, PostgreSQL, Grafana e Docker.

A UC de ISI incide sobre a integração de sistemas de informação ao nível dos dados. Neste contexto, os processos de ETL assumem um papel central: permitem recolher dados provenientes de origens e formatos heterogéneos, transformá-los de acordo com regras de negócio e disponibilizá-los em sistemas de destino, onde podem ser consultados, analisados e visualizados. Como refere o enunciado, a reconfiguração constante dos processos de negócio e a necessidade de rentabilizar investimentos anteriores fazem da migração, da reorganização e da análise contínua de dados cenários onde o ETL desempenha um papel preponderante (Ferreira & Ribeiro, 2026).

O enunciado exige que o processo desenvolvido se estruture em três fases obrigatórias: recolha e preparação de dados (fase «E»), processamento e transformação de dados (fase «T») e persistência e exportação (fase «L»). O tema escolhido para demonstrar estas três fases foi a análise do poder de compra no setor tecnológico, cujo problema se descreve na secção seguinte.

1.2. Descrição do Problema e sua Relevância

A procura de emprego na área das tecnologias de informação é, atualmente, um mercado global. Um profissional pode considerar ofertas em dezenas de países, mas comparar essas ofertas de forma rigorosa é uma tarefa difícil, porque os dados disponíveis nos anúncios são heterogéneos e, frequentemente, inconsistentes. Identificam-se quatro dificuldades principais:

- **Moedas distintas:** os salários são expressos em libras, euros, dólares ou outras moedas, não sendo diretamente comparáveis sem conversão cambial;
- **Periodicidades distintas:** o mesmo campo salarial pode representar um valor por hora, por dia, por mês ou por ano — um valor de 45 pode corresponder a um salário horário, 3 500 a um salário mensal e 50 000 a um salário anual;
- **Localizações inconsistentes:** o campo de localização mistura cidades, regiões, países e códigos postais, o que impede a agregação por país sem um tratamento prévio do texto;
- **Salário bruto versus poder de compra:** o salário bruto anunciado não reflete a capacidade real de poupança, que depende dos impostos, do custo da habitação (renda) e do custo de vida do país onde a oferta se localiza.

O problema que o trabalho procura resolver pode, assim, ser formulado através da seguinte questão: *em que mercados tecnológicos pode um profissional, de facto, poupar mais ao longo de um ano, depois de normalizadas as moedas, as periodicidades e as localizações e de descontados os impostos e o custo de vida?* A relevância do problema é prática — apoia decisões de carreira e de mobilidade internacional, bem como a análise de mercado por parte de recrutadores — e é também técnica, porque constitui um cenário realista onde se tornam indispensáveis a limpeza, a normalização e o enriquecimento de dados antes da sua análise.

1.3. Motivação e Objetivos

A motivação para o tema resulta da possibilidade de trabalhar com dados reais, obtidos através de *Application Programming Interfaces* (API) *Representational State Transfer* (REST), de ficheiros históricos e de páginas web, e de os integrar numa arquitetura que cobre todo o ciclo de vida dos dados. O tema permite ainda explorar os critérios de mais-valia indicados no enunciado: diversidade de formatos, expressões regulares (ER), junções e enriquecimento por *lookup*, acesso a APIs remotas,

orquestração de *jobs*, operações sobre bases de dados (BD), notificações automáticas e visualização em *dashboards*.

O **objetivo geral** consiste em desenvolver um *pipeline* de ETL automático e reprodutível que recolha ofertas de emprego tecnológicas de origens heterogéneas, as normalize e enriqueça com indicadores de custo de vida, e as disponibilize num painel analítico interativo e num canal de notificações. Como **objetivos específicos** definiram-se os seguintes:

- consumir dados em três formatos distintos — *JavaScript Object Notation* (JSON), *Comma-Separated Values* (CSV) e *HyperText Markup Language* (HTML) — a partir de APIs REST, de um ficheiro local e de uma página web;
- limpar e validar os dados, descartando registos sem título ou sem salário, e normalizar a localização com recurso a ER e a geocodificação por código postal;
- anualizar os salários, independentemente da periodicidade original, e convertê-los para euros;
- cruzar as ofertas com os índices de custo de vida da Numbeo e calcular a Poupança Anual Estimada;
- orquestrar duas vias de processamento complementares — uma em lote (*batch*) e outra contínua (*streaming*) — desacoplando esta última através de uma fila de mensagens assíncrona;
- persistir os resultados numa BD relacional PostgreSQL, com modelos de escrita distintos para cada via;
- construir um *dashboard* no Grafana com filtros dinâmicos e um sistema de alertas automáticos via Telegram;
- garantir a reprodutibilidade do ambiente através de contentores Docker.

1.4. Metodologia e Estrutura do Documento

A abordagem seguida organizou-se nas três fases do ETL previstas no enunciado e concretizou-se em duas vias de processamento complementares, descritas no Capítulo 2. A Tabela 1 sintetiza a correspondência entre as fases, os componentes tecnológicos e os capítulos onde cada uma é documentada.

Fase	Componentes	Operações principais	Capítulos
E – Recolha e preparação	KNIME (GET Request, CSV Reader, Webpage Retriever); Node-RED (http request)	Pedidos a APIs REST, leitura de ficheiro CSV, *web scraping* com XPath, divisão de resultados JSON	2 e 3
T – Processamento e transformação	KNIME (Rule Engine, Math Formula, Joiner); Node-RED (nós *function*)	Limpeza, ER, anualização, conversão cambial, *lookup*, cálculo da poupança e mapeamento de *schema*	3
L – Persistência e exportação	KNIME (DB Writer); RabbitMQ; PostgreSQL; Grafana; Telegram	Fila assíncrona, escrita em lote, *upsert*, agregações SQL, visualização e notificações	4 e 5

Tabela 1: Correspondência entre as fases do ETL, os componentes e os capítulos do documento.

O documento está organizado em oito capítulos, que acompanham os pontos exigidos para o relatório no enunciado, conforme se indica na Tabela 2.

N.º	Ponto do enunciado	Localização
1	Enquadramento (equipa, UC, curso)	Secção 1.1
2	Problema	Secções 1.2 e 1.3
3	Estratégia utilizada	Capítulo 2
4	Transformações (diagramas e explicação)	Capítulo 3
5	*Jobs* (diagramas e explicação)	Capítulos 4 e 5
6	Vídeo com demonstração (*QR Code*)	Capítulo 6
7	Conclusão e trabalhos futuros	Capítulo 7
8	Referências bibliográficas e web	Capítulo 8

Tabela 2: Correspondência entre os pontos exigidos no enunciado e a sua localização neste documento.

2. Estratégia Utilizada e Arquitetura

Este capítulo descreve a estratégia adotada para resolver o problema — isto é, que operadores e processos estão envolvidos — e a arquitetura que os suporta. Apresenta-se a arquitetura híbrida em duas vias, o ambiente contentorizado, a justificação das escolhas tecnológicas e a forma como o projeto cobre as três fases obrigatórias do enunciado.

2.1. Arquitetura Híbrida: Processamento em Lote e Contínuo

A solução adota uma arquitetura complementar, composta por duas vias de processamento que convergem para a mesma BD analítica. A **via em lote** (*batch* ETL), implementada no KNIME Analytics Platform, é responsável pelo processamento analítico pesado: consome três formatos de dados distintos, cruza as ofertas com indicadores macroeconómicos e calcula a poupança líquida por país. A **via contínua** (*streaming* ETL), implementada no Node-RED com RabbitMQ, funciona como sonda de baixa latência: recolhe ofertas de hora a hora, aplica um tratamento leve e despacha alertas imediatos. A Figura 1 apresenta a visão global da arquitetura e do percurso dos dados.

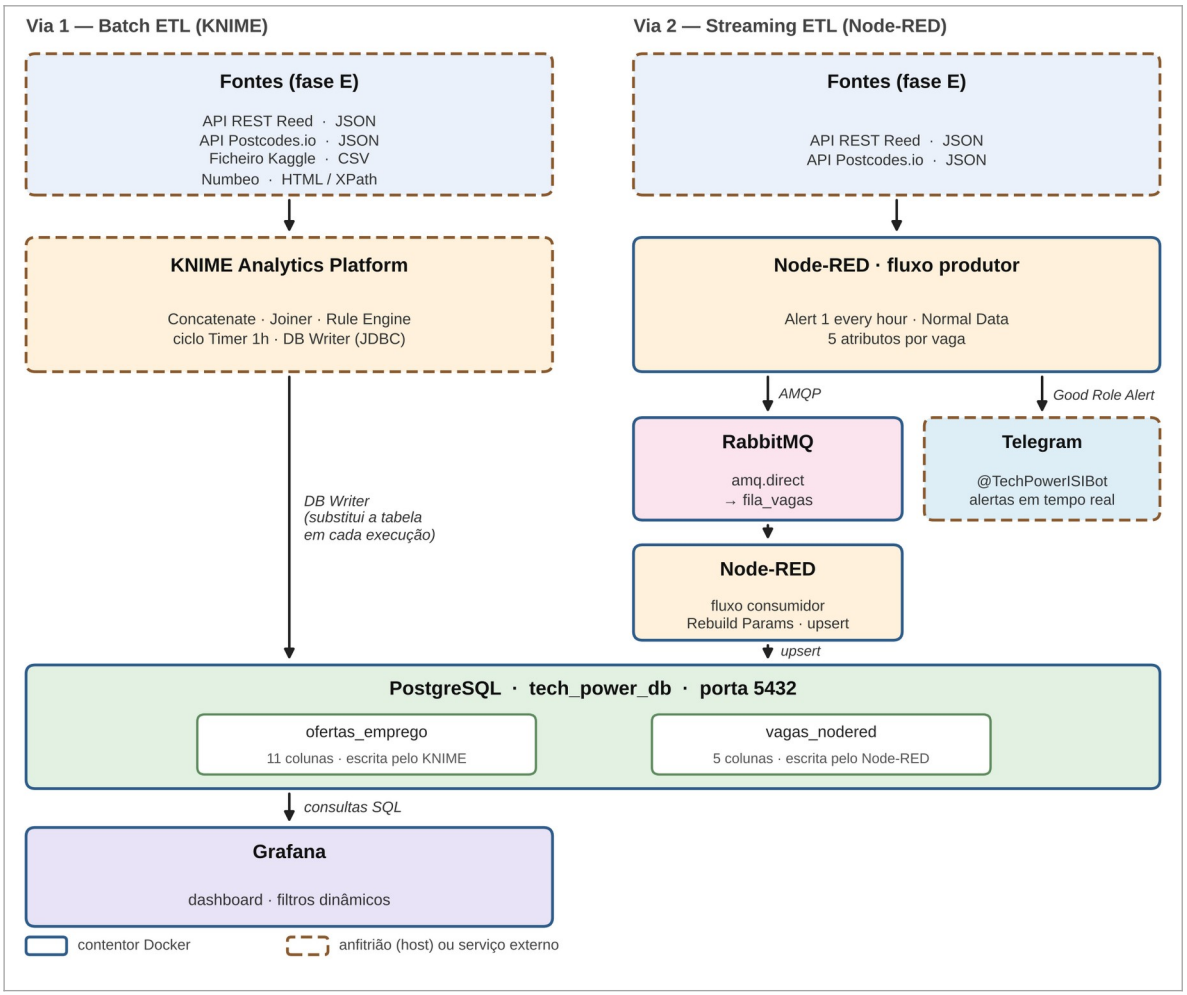


Figura 1: Arquitetura híbrida do pipeline ETL Tech Purchasing Power.

As duas vias não são redundantes: respondem a necessidades diferentes e escrevem em tabelas distintas da mesma BD, como se detalha na secção 4.4. A Tabela 3 resume as diferenças entre ambas.

Característica	Via 1 — Batch (KNIME)	Via 2 — Streaming (Node-RED)
Objetivo	Análise macroeconómica e carga histórica massiva	Monitorização de novas ofertas e alertas imediatos
Formatos de entrada	JSON, CSV e HTML/XPath	JSON
Origens	API Reed, API Postcodes.io, ficheiro Kaggle, página Numbeo	API Reed, API Postcodes.io
Enriquecimento	Índices Numbeo e taxa de imposto por país	Não aplicável (tratamento leve)
Desacoplamento	Não aplicável (execução direta)	Fila de mensagens RabbitMQ (AMQP)
Tabela de destino	`ofertas_emprego` (11 colunas)	`vagas_nodered` (5 colunas)
Modo de escrita	Substituição da tabela em cada execução	*Upsert* com deduplicação
Saída final	Dashboard analítico no Grafana	Alertas no Telegram e monitorização

Tabela 3: Comparação entre as duas vias de processamento do pipeline.

2.2. Ambiente Contentorizado (Docker)

Os serviços de suporte foram executados em contentores Docker, geridos através do Docker Desktop (Docker, s.d.). A contentorização assegura o isolamento de cada serviço, a reprodutibilidade do ambiente em qualquer máquina e a simplicidade de arranque e paragem de toda a solução. Os serviços comunicam entre si através da rede interna criada pelo Docker, expondo ao anfitrião apenas as portas necessárias ao acesso às respetivas interfaces. A Figura 2 apresenta os quatro contentores em execução e a Tabela 4 resume a função de cada um.

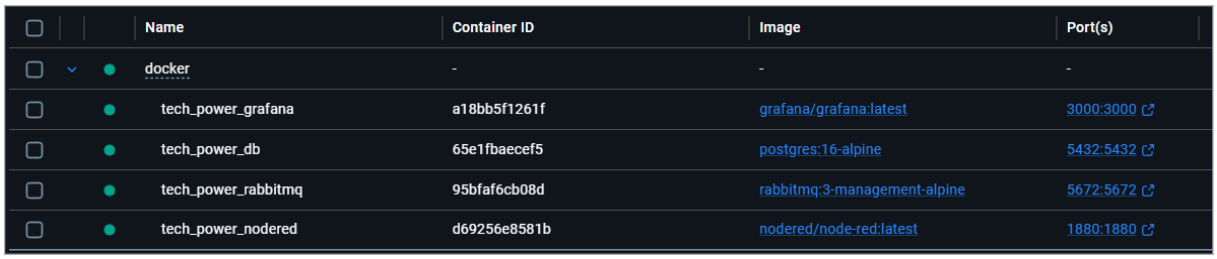


Figura 2: Contentores do pipeline em execução no Docker Desktop.

Contentor	Imagem	Porta	Função
`tech_power_nodered`	`nodered/node-red:latest`	1880:1880	Via contínua: extração, transformação, consumo da fila e alertas
`tech_power_rabbitmq`	`rabbitmq:3-management-alpine`	5672:5672	Fila de mensagens que desacopla produtor e consumidor
`tech_power_db`	`postgres:16-alpine`	5432:5432	Base de dados relacional PostgreSQL (destino das duas vias)
`tech_power_grafana`	`grafana/grafana:latest`	3000:3000	Visualização analítica

Tabela 4: Contentores que compõem o ambiente de execução.

O KNIME Analytics Platform constitui a exceção a este ambiente: por se tratar de uma aplicação de *desktop* com interface gráfica, é executado diretamente no anfitrião, acedendo ao PostgreSQL através da porta 5432 publicada pelo contentor `tech_power_db`. O Telegram, por ser um serviço externo, é igualmente alcançado a partir do anfitrião através da Internet.

2.3. Justificação das Escolhas Tecnológicas

A seleção de cada componente foi orientada pelas necessidades do problema:

- **KNIME Analytics Platform** — ferramenta de ETL visual, expressamente sugerida no enunciado, que disponibiliza nós nativos para leitura de CSV, pedidos HTTP, *parsing* de JSON, extração de HTML por XPath, junções, regras de negócio e escrita em BD, permitindo construir um fluxo multi-formato sem programação explícita (KNIME, s.d.);
- **Node-RED** — ferramenta de programação baseada em fluxos, orientada a eventos, adequada à recolha periódica e ao encaminhamento de mensagens, com nós para HTTP, AMQP, PostgreSQL e execução de JavaScript (Node-RED, s.d.);
- **RabbitMQ** — *broker* de mensagens baseado no protocolo *Advanced Message Queuing Protocol* (AMQP), escolhido para desacoplar a extração da persistência: se a BD estiver temporariamente indisponível ou lenta, as mensagens permanecem na fila, sem perda de dados e sem bloquear a recolha (RabbitMQ, s.d.);
- **PostgreSQL 16** — sistema de gestão de bases de dados relacional, adequado a esquemas estáveis e a agregações analíticas em *Structured Query Language* (SQL), como contagens, mínimos, médias, máximos e agrupamentos, e que

suporta nativamente a cláusula `ON CONFLICT` usada na deduplicação (PostgreSQL Global Development Group, s.d.);

- **Grafana** — plataforma de visualização com ligação nativa ao PostgreSQL e suporte para variáveis dinâmicas, que permitem filtrar todos os painéis em simultâneo (Grafana Labs, s.d.);
- **Telegram Bot API** — canal de notificação gratuito e de integração simples por pedido HTTP, usado para concretizar o requisito de alertas automáticos previsto no enunciado (Telegram, s.d.);
- **Docker** — contentorização e orquestração dos serviços numa rede interna (Docker, s.d.).

2.4. Evolução da Solução e Abordagens Exploradas

A arquitetura apresentada não foi definida à partida: resultou de um processo iterativo, em que várias abordagens foram ensaiadas, avaliadas e, nalguns casos, abandonadas. Documentam-se nesta secção as decisões mais relevantes e as respetivas razões, por constituírem parte do trabalho desenvolvido e por justificarem a forma final da solução. A Tabela 5 resume o percurso.

Abordagem ensaiada	Limitação encontrada	Decisão final
Node-RED como única ferramenta de ETL	Enriquecimento limitado a constantes escritas no código, para dois países apenas; ausência de nós nativos para CSV e *web scraping*	Introdução do KNIME para o processamento em lote multi-formato, mantendo o Node-RED para a ingestão contínua
Índices de custo de vida como constantes no nó Normal Data	Valores fixos e desatualizados, limitados ao Reino Unido e à Irlanda	Extração por XPath da página do Numbeo, abrangendo os dez mercados analisados
MongoDB como destino do fluxo do Node-RED	Limitações do plano gratuito; ligação ao Grafana exigiria componente adicional	Segunda tabela no PostgreSQL (`vagas_nodered`), isolada da tabela do KNIME
Tabela única partilhada pelas duas vias	O modo de substituição do nó DB Writer apagava os registos inseridos em tempo real	Separação em duas tabelas com modelos de escrita distintos
Inserção simples (`INSERT`) na via contínua	Acumulação de registos duplicados a cada execução horária	Restrição de unicidade e escrita por cima (`ON CONFLICT DO UPDATE`)
Execução manual do *workflow* do KNIME	Dependência de intervenção do utilizador; desalinhamento face à via contínua	Encapsulamento num ciclo com nó *Wait* de uma hora

Tabela 5: Abordagens exploradas ao longo do desenvolvimento e decisão final.

2.4.1. Tentativa de Diversificação com MongoDB

Uma das linhas exploradas com maior persistência foi a introdução de uma BD *Not only SQL* (NoSQL), concretamente o MongoDB, como destino do fluxo do Node-RED (MongoDB, s.d.). A motivação era dupla. Por um lado, **diversificar as tecnologias de persistência**, demonstrando a integração de dois paradigmas distintos de armazenamento no mesmo *pipeline* — um relacional e um orientado a documentos — o que acrescentaria valor ao trabalho enquanto exercício de integração de sistemas. Por outro, o modelo documental afigurava-se adequado à via contínua: as ofertas chegam como documentos JSON e poderiam ser armazenadas sem conversão prévia para um esquema tabular fixo.

A abordagem foi abandonada por **limitações do plano gratuito** do serviço, que condicionaram a dimensão do armazenamento e o número de operações disponíveis, inviabilizando a ingestão horária continuada pretendida. Acresce que a ligação do Grafana ao MongoDB não é nativa, ao contrário do que sucede com o PostgreSQL, exigindo um componente intermédio adicional que aumentaria a complexidade da arquitetura sem benefício analítico correspondente.

A alternativa adotada preserva parcialmente o objetivo inicial: em vez de duas tecnologias de BD, mantêm-se **duas tabelas com modelos de escrita distintos** na mesma instância PostgreSQL — uma escrita em lote por substituição e outra escrita continuamente por *upsert* —, o que permite na mesma demonstrar a convivência de duas cadências e dois modelos de persistência no mesmo destino. A diversificação tecnológica acabou por ser assegurada noutras dimensões do trabalho: na extração, com três formatos e quatro origens distintas, e na exportação, com *dashboard* analítico e notificações em tempo real.

2.4.2. Justificação da Opção Final pelo PostgreSQL

A adoção final do PostgreSQL assenta em três razões: (i) as ofertas, depois de normalizadas, têm um esquema estável e tabular, que não beneficia da flexibilidade de um modelo documental; (ii) as consultas necessárias ao *dashboard* são, na sua essência, agregações e filtros que se exprimem de forma natural em SQL; e (iii) o Grafana oferece uma fonte de dados PostgreSQL nativa, com suporte a variáveis dinâmicas baseadas em consultas `SELECT DISTINCT`, o que simplifica substancialmente a construção dos filtros.

2.5. Cobertura das Fases do ETL

2.5.1. Fase 1 – Recolha e Preparação de Dados (E)

O enunciado exige o consumo de dados em pelo menos dois ou três formatos distintos e a partir de origens de natureza variada. A Tabela 6 demonstra o cumprimento deste requisito, identificando para cada origem o formato, o mecanismo de acesso e a via que a consome.

Origem	Formato	Mecanismo de acesso	Via
API REST da Reed	JSON	Pedidos HTTP GET a uma API REST remota	Batch e contínua
API Postcodes.io	JSON	Pedidos HTTP GET de geocodificação	Batch e contínua
Conjunto de dados Kaggle	CSV	Leitura de ficheiro local (*CSV Reader*)	Batch
Numbeo *Cost of Living*	HTML	*Web scraping* com expressões XPath	Batch
PostgreSQL	Tabela relacional	Consultas SQL a partir do Grafana	Leitura analítica

Tabela 6: Formatos e origens de dados consumidos pelo pipeline.

2.5.2. Fase 2 – Processamento e Transformação de Dados (T)

Na fase «T» foram aplicadas as operações que constituem o núcleo do trabalho: (i) **limpeza e validação**, com a remoção de registos sem título, com salário nulo ou com valores mal introduzidos; (ii) utilização de **expressões regulares** e validação textual para separar e normalizar localizações; (iii) **conversão de estruturas**, de documentos JSON e de árvores HTML para tabelas e, no Node-RED, de objeto JSON para *array* parametrizado; (iv) **cruzamento e enriquecimento** através de concatenação vertical (*Concatenate*) e de junção relacional (*Joiner*) com os índices Numbeo; e (v) **cálculos matemáticos** de anualização salarial, conversão cambial, dedução fiscal e poupança líquida. Estas operações são documentadas em detalhe no Capítulo 3.

2.5.3. Fase 3 – Persistência e Exportação (L)

Na fase «L» os dados transformados seguem três destinos complementares: a **persistência** em duas tabelas do PostgreSQL, com modos de escrita distintos; a **visualização dinâmica** em *dashboards* interativos no Grafana, que se atualizam em

função dos filtros selecionados; e a **notificação automática** de ofertas relevantes através de um *bot* do Telegram. Esta fase é documentada nos Capítulos 4 e 5.

3. Transformações

Este capítulo documenta as fases «E» e «T» em ambas as vias de processamento. Apresentam-se os diagramas dos fluxos, a explicação de cada nó e as regras de negócio aplicadas.

3.1. Via em Lote: Workflow no KNIME Analytics Platform

O *workflow* desenvolvido no KNIME organiza-se em três ramos paralelos de extração, que convergem para um tronco comum de padronização, cruzamento e cálculo, terminando na escrita em BD. Dada a sua extensão horizontal, o *workflow* é apresentado em duas metades: a Figura 3 mostra os três ramos de extração e respetiva preparação, e a Figura 4 mostra a convergência, o enriquecimento e a persistência. As subsecções seguintes detalham cada zona através de ampliações.

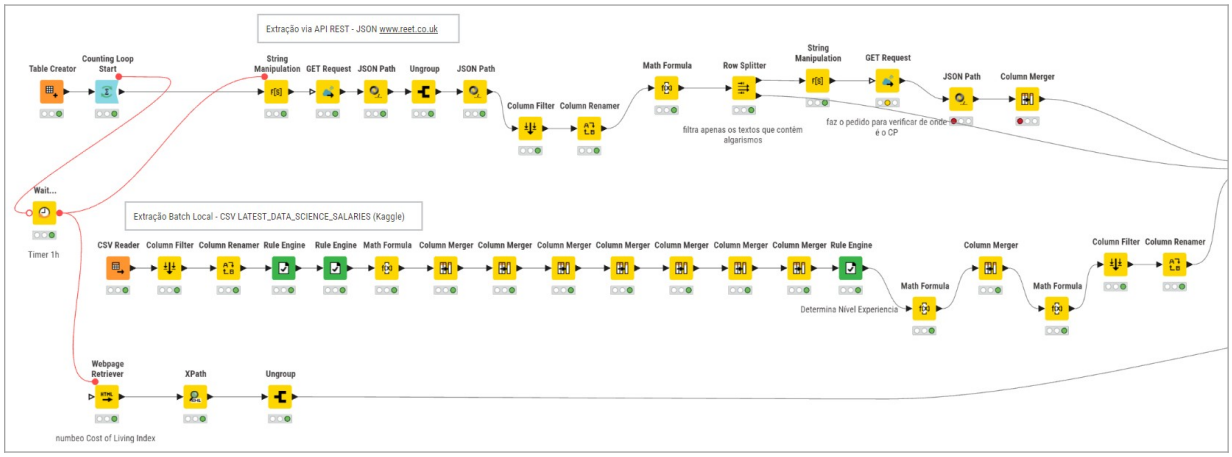


Figura 3: Workflow KNIME (1 de 2): ramos de extração e preparação.

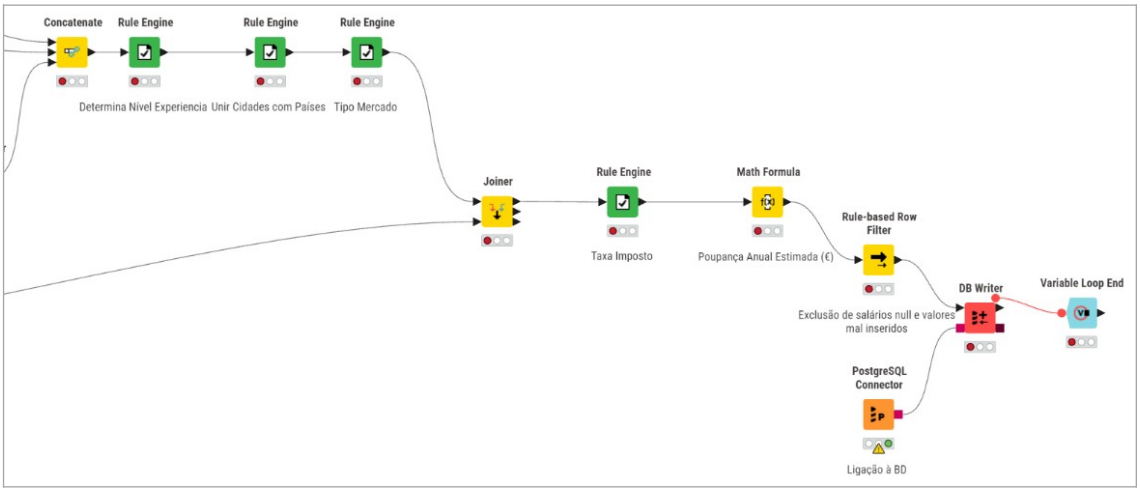


Figura 4: Workflow KNIME (2 de 2): fusão, enriquecimento e persistência.

3.1.1. Ramo 1 – Extração via API REST (JSON)

O primeiro ramo, ampliado na Figura 5 a partir da Figura 3, recolhe as ofertas de emprego a partir da API REST da Reed (Reed, s.d.). O nó **Table Creator** define a tabela inicial com as palavras-chave de pesquisa tecnológica e o nó **String Manipulation** constrói dinamicamente os *Uniform Resource Locators* (URL) de chamada, concatenando o *endpoint* base com cada termo e com os parâmetros de paginação.

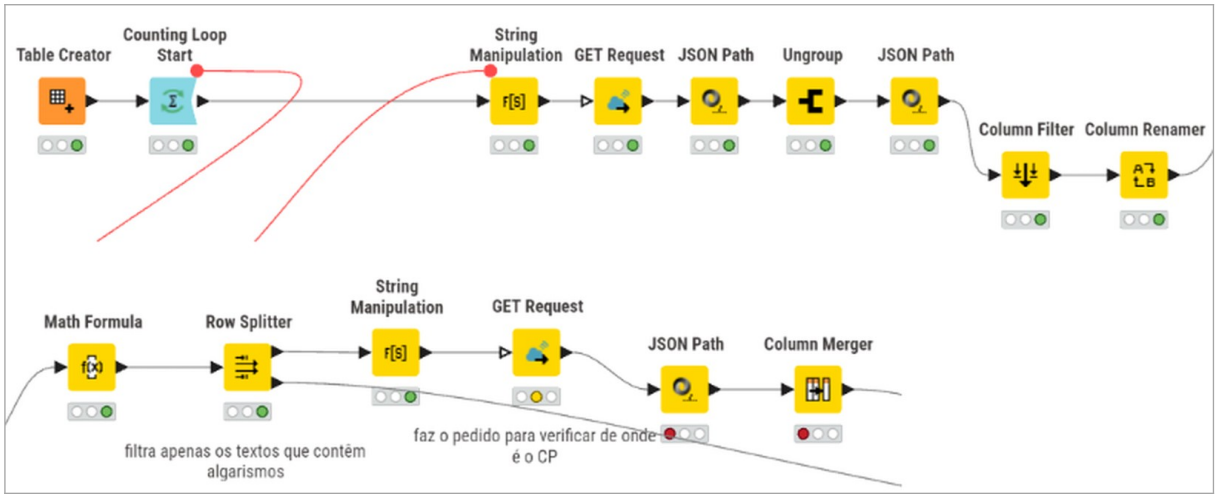


Figura 5: Ramo de extração via API REST e resolução de códigos postais (KNIME).

O nó **GET Request** executa os pedidos HTTP e devolve as respostas em JSON. A sequência **JSON Path** → **Ungroup** → **JSON Path** extrai a lista de ofertas da estrutura devolvida, desagrupa o *array* de modo a obter uma linha por oferta e extrai os atributos de cada vaga (título, localização e intervalo salarial). Os nós **Column Filter** e **Column Renamer** removem as colunas acessórias devolvidas pela API e padronizam os nomes dos atributos para o esquema canónico do *pipeline*.

Segue-se o tratamento salarial e geográfico. O nó **Math Formula** calcula a remuneração base e aplica a conversão cambial de libras para euros. O nó **Row Splitter**, anotado no *workflow* com a indicação «filtra apenas os textos que contêm algarismos», aplica um critério baseado em padrões para separar os registos cuja localização contém algarismos — isto é, códigos postais — daqueles que já apresentam nomes de localidades legíveis. Os primeiros seguem para um sub-ramo de geocodificação, composto por **String Manipulation** (construção do URL de consulta), **GET Request** (anotado com «faz o pedido para verificar de onde é o CP»), **JSON Path** (extração do país) e **Column Merger** (fusão do país obtido na coluna de

localização). Os registos assim resolvidos juntam-se novamente aos restantes no nó *Concatenate*.

3.1.2. Ramo 2 – Extração em Lote de Ficheiro Local (CSV)

O segundo ramo, ampliado na Figura 6, importa o conjunto de dados `LATEST_DATA_SCIENCE_SALARIES`, obtido a partir do Kaggle (Kaggle, s.d.), através do nó **CSV Reader**. Este ficheiro contém o histórico global de salários nas áreas de *data science* e tecnologias de informação e é a origem do volume que sustenta a análise comparativa entre mercados, abrangendo os dez países que surgem no *dashboard* final.

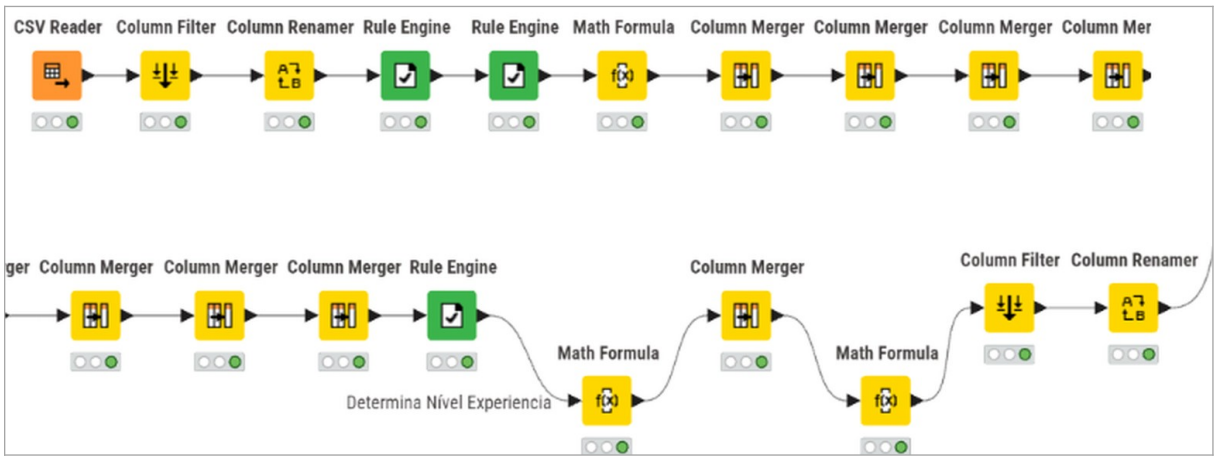


Figura 6: Ramo de extração e normalização do ficheiro CSV (KNIME).

Os nós **Column Filter** e **Column Renamer** selecionam as colunas relevantes e uniformizam a nomenclatura, de modo a coincidir com a estrutura do primeiro ramo. Segue-se uma cadeia de nós **Rule Engine**, **Math Formula** e sucessivos **Column Merger** que normaliza as moedas e as escalas salariais e consolida colunas fragmentadas. O nó **Rule Engine** anotado com «Determina Nível Experiencia» converte os códigos de senioridade do conjunto de dados original (por exemplo, `EN`, `MI`, `SE` e `EX`) nas três categorias padronizadas utilizadas no *pipeline*: *Entry*, *Mid* e *Senior*. Os nós finais de **Math Formula**, **Column Filter** e **Column Renamer** concluem a conversão salarial para euros, eliminam as colunas temporárias de cálculo e alinham o esquema com o do primeiro ramo, preparando a concatenação.

3.1.3. Ramo 3 – Extração Web com XPath (HTML)

O terceiro ramo, ampliado na Figura 7, é responsável pela recolha dos indicadores macroeconómicos. O nó **Webpage Retriever**, anotado com «numbeo

Cost of Living Index», efetua o pedido HTTP à página pública do Numbeo e recolhe o documento em HTML (Numbeo, s.d.).

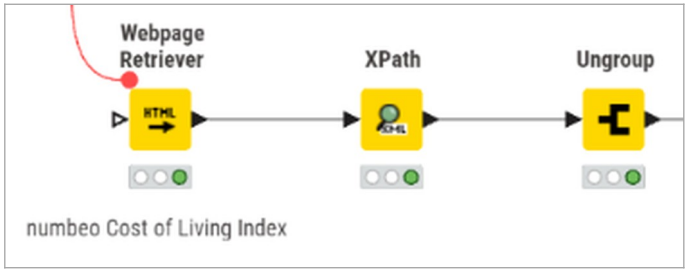


Figura 7: Ramo de extração dos índices económicos do Numbeo por XPath (KNIME).

O nó **XPath** aplica expressões sobre a árvore *Document Object Model* (DOM) do documento para extrair as colunas da tabela de índices por país, nomeadamente o `Indice_Custo_Vida`, o `Indice_de_Renda` e o `Indice_Custo_Vida_Mais_Renda`. O nó **Ungroup** desagrupa as listas resultantes em linhas tabulares, obtendo-se uma tabela de referência com uma linha por país, pronta a ser cruzada com as ofertas. Esta abordagem substitui com vantagem a introdução manual de constantes no código: os índices passam a ser recolhidos diretamente da fonte e abrangem todos os países presentes no conjunto de dados.

3.1.4. Fusão e Padronização Global

O nó **Concatenate**, visível na Figura 8, une verticalmente as ofertas provenientes da API REST (ramo 1) e do ficheiro CSV (ramo 2) numa tabela única. Esta operação só é possível porque ambos os ramos foram previamente alinhados ao mesmo esquema de colunas.



Figura 8: Concatenação e regras de padronização global (KNIME).

Seguem-se três nós **Rule Engine** aplicados em cadeia à tabela consolidada:

- **Determina Nível Experiencia** — uniformiza o nível de experiência dos registos provenientes da API, que não dispõem dessa classificação de origem, atribuindo-lhes uma das três categorias padronizadas;

- **Unir Cidades com Países** — mapeia nomes de cidades para os respetivos países, garantindo que a nomenclatura coincide exatamente com a utilizada na tabela de índices do Numbeo, o que é condição necessária para a junção seguinte;
- **Tipo Mercado** — classifica cada registo quanto ao tipo de mercado a que pertence.

3.1.5. Cruzamento, Modelação Fiscal e Cálculo da Poupança

A Figura 9 amplia o tronco final do *workflow*, já visível na Figura 4. O nó **Joiner** realiza a junção relacional entre o fluxo consolidado de ofertas (porta superior) e a tabela de índices económicos extraída por XPath (porta inferior), usando o país como chave de ligação. Desta operação resulta a associação, a cada oferta, dos três índices do respetivo país.

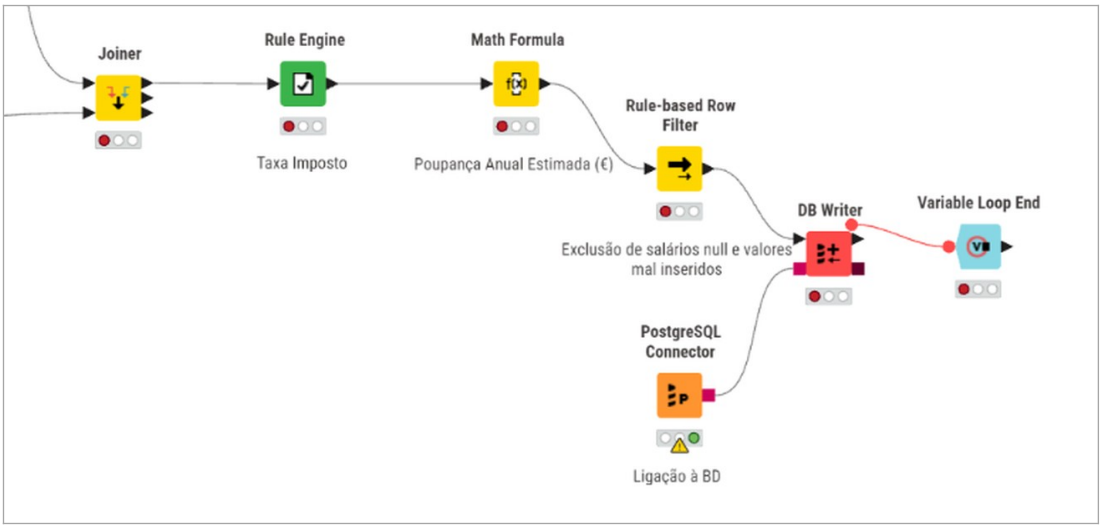


Figura 9: Junção com os índices Numbeo, cálculo da poupança e escrita em base de dados (KNIME).

O nó **Rule Engine** anotado com «Taxa Imposto» atribui a taxa de imposto aplicável a cada registo, em função do país e do escalão salarial, tendo sido adotada uma taxa estimada de 30 % como valor de referência. O nó **Math Formula**, anotado com «Poupança Anual Estimada (€)», executa o cálculo final em duas etapas conceptuais: primeiro deduz o imposto ao salário bruto anualizado em euros, obtendo o rendimento líquido anual; em seguida pondera esse rendimento face ao custo de vida e de habitação do país, representado pelos índices do Numbeo, obtendo a capacidade de poupança efetiva. O indicador resultante traduz, assim, o poder de compra real da oferta no país de destino, e não apenas o valor nominal anunciado.

Por fim, o nó **Rule-based Row Filter**, anotado com «Exclusão de salários null e valores mal inseridos», atua como filtro final de qualidade dos dados, eliminando linhas com valores nulos, salários iguais ou inferiores a zero e anomalias numéricas, antes da escrita na BD. A persistência propriamente dita, assegurada pelos nós **PostgreSQL Connector** e **DB Writer**, é documentada na secção 4.4.

3.2. Via Contínua: Transformações no Node-RED

A Figura 10 apresenta o fluxo produtor implementado no Node-RED. O fluxo é iniciado pelo nó de disparo periódico *Alert 1 every hour*, descrito na secção 4.2, e percorre em sequência a preparação dos termos de pesquisa, a chamada às APIs e o tratamento dos dados. Observa-se ainda, na parte inferior do diagrama, a bifurcação do nó *Normal Data* para o ramo de notificações (nós *Good Role Alert* e *http request Telegram*), documentado na secção 4.5.

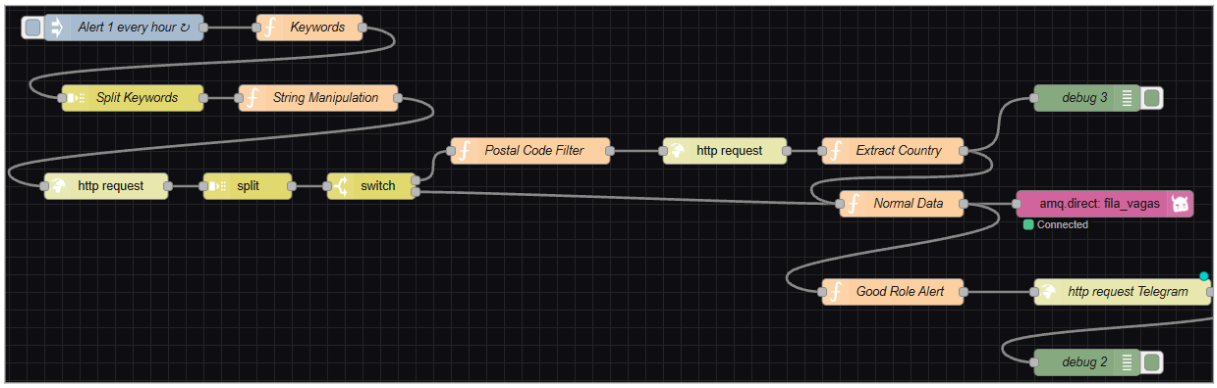


Figura 10: Fluxo produtor de extração e transformação no Node-RED.

3.2.1. Preparação dos Termos e Chamadas às APIs

O nó **Keywords** gera a lista de termos de pesquisa de natureza tecnológica e o nó **Split Keywords** separa essa lista em mensagens individuais, de modo que cada termo seja processado de forma independente. O nó **String Manipulation** formata cada termo para construir o parâmetro de pesquisa do *endpoint*. O primeiro nó **http request** efetua o pedido à API da Reed e o nó **split** divide o *array* JSON de resultados em mensagens individuais, de forma que cada oferta percorra o restante fluxo como uma mensagem autónoma.

O nó **switch** avalia o campo de localização e encaminha a mensagem por uma de duas saídas. Quando a localização corresponde a um código postal, a mensagem segue para o nó **Postal Code Filter**, que isola e valida o código com uma expressão regular, e daí para um segundo nó **http request**, que consulta a API Postcodes.io

(Postcodes.io, s.d.); o nó **Extract Country** extrai o país da resposta. Caso contrário, a mensagem segue diretamente para o nó **Normal Data**. Como exemplo ilustrativo, os códigos postais do Reino Unido seguem um padrão do tipo `^[A-Z]{1,2}\d[A-Z\d]?\s*\d[A-Z]{2}$`, que pode ser testado diretamente em JavaScript (MDN Web Docs, s.d.).

3.2.2. Regras de Negócio no Nó Normal Data

O nó **Normal Data** é um nó *function* que concentra a lógica de negócio desta via e cuja saída se bifurca em duas vias paralelas: a publicação na fila do RabbitMQ e a avaliação de notificações, documentada na secção 4.5. As regras aplicadas são as seguintes.

Filtro de qualidade (*gatekeeper*). Descartam-se as ofertas sem título e as ofertas cujo salário é zero, por não possuírem informação suficiente. Ao devolver `null`, o nó termina o processamento da mensagem, que não chega a ser publicada na fila.

```
// Filtro de qualidade
if (!vaga.jobTitle || salario_libras === 0) {
    return null;   // descarta o registo
}
```

Anualização salarial e conversão cambial. Como o campo salarial pode representar valores com periodicidades diferentes, a regra infere a periodicidade a partir da ordem de grandeza e converte o valor para um salário anual: valores inferiores a 100 são interpretados como salário horário e multiplicados por 1 950; valores entre 100 e 1 500 como salário diário e multiplicados por 260; e valores entre 1 500 e 10 000 como salário mensal e multiplicados por 12. O resultado é convertido para euros através da taxa de câmbio de 1,17.

```
// Anualização e conversão cambial
let salario_anual = salario_libras;

if (salario_libras < 100) {
    salario_anual = salario_libras * 1950;   // salário/hora
} else if (salario_libras < 1500) {
    salario_anual = salario_libras * 260;    // salário/dia
} else if (salario_libras < 10000) {
    salario_anual = salario_libras * 12;     // salário/mês
}
const salario_euros = Math.round(salario_anual * 1.17);
```

Normalização geográfica. O país é fixado em *United Kingdom* por omissão, sendo alterado para *Ireland* apenas quando o campo de localização contém uma das palavras *ireland*, *dublin*, *cork* ou *galway*. A comparação é feita sobre o texto em minúsculas e através de uma expressão regular, o que torna a validação independente

de maiúsculas e de variações de escrita. Esta normalização é deliberadamente mais simples do que a da via em lote, uma vez que esta via se destina à deteção rápida de novas ofertas e não à análise comparativa entre mercados.

```
// Normalização geográfica
const local = (vaga.locationName || '').toLowerCase();
let pais = 'United Kingdom';

if (/ireland|dublin|cork|galway/.test(local)) {
    pais = 'Ireland';
}
```

Mapeamento de *schema*. A oferta é reestruturada, de objeto JSON para um *array* de cinco posições, por ordem fixa, correspondente às colunas da tabela de destino descrita na secção 4.4. Os campos de nível de experiência e de tipo de mercado assumem valores fixos nesta via, por não existir informação de origem que permita determiná-los. O *array* é atribuído ao *payload* da mensagem e enviado para o nó de publicação no RabbitMQ, onde é serializado em JSON.

```
// Mapeamento de schema (5 atributos, por ordem)
msg.payload = [
    vaga.jobTitle,                // 0  titulo_vaga
    pais,                         // 1  localizacao (país normalizado)
    salario_euros,                // 2  salario_euros
    'Mid',                        // 3  nivel_experiencia
    'Mercado Local / Standard'    // 4  tipo_mercado
];
return msg;
```

3.2.3. Reconstrução e Validação de Parâmetros (Rebuild Params)

No fluxo consumidor, descrito na secção 4.3, a mensagem recebida do RabbitMQ chega sob a forma de um pacote binário (*Buffer*). O nó **Rebuild Params** converte esse pacote para texto e interpreta-o como JSON. Para além da conversão, o nó valida que a estrutura recebida é efetivamente um *array* e injeta valores por omissão em cada posição em falta, o que impede que mensagens truncadas ou respostas inválidas das APIs provoquem erros na escrita em BD.

```
// Rebuild Params
const dados = JSON.parse(msg.payload.toString());

if (!Array.isArray(dados)) { return null; }

msg.params = [
    dados[0] || 'Sem Titulo',
    dados[1] || 'Desconhecido',
    dados[2] || 0,
    dados[3] || 'N/A',
    dados[4] || 'N/A'
];
return msg;
```

4. Jobs e Persistência de Dados

Este capítulo documenta os *jobs* que orquestram o *pipeline* — o ciclo de execução do KNIME, o agendamento horário do Node-RED, a mensageria assíncrona, o carregamento em BD e o despacho de notificações — bem como o modelo de dados adotado. Estes *jobs* constituem o núcleo da fase «L».

4.1. Job de Execução Cíclica no KNIME

Para que a via em lote não dependesse de execuções manuais, o *workflow* do KNIME foi encapsulado num ciclo de repetição, visível na Figura 11. O nó **Counting Loop Start**, colocado imediatamente após o `Table Creator`, abre o ciclo e define o número de iterações a executar; o nó **Variable Loop End**, no extremo oposto do *workflow* (ver Figura 4), fecha-o e devolve o controlo ao início. Entre ambos, o nó **Wait**, anotado com «Timer 1h», suspende a execução durante uma hora antes de cada nova iteração.

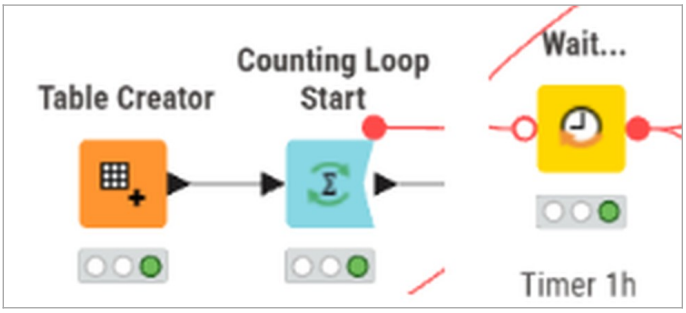


Figura 11: Nós de controlo do ciclo de execução periódica no KNIME.

Deste modo, o *workflow* repete integralmente a extração, a transformação e a escrita em BD a cada hora, alinhando a cadência da via em lote com a da via contínua. As ligações a vermelho no *workflow* representam o fluxo de variáveis de controlo do ciclo, distinto do fluxo de dados representado a cinzento.

4.2. Job de Agendamento e Mensageria Assíncrona (RabbitMQ)

Na via contínua, o *job* de extração é disparado automaticamente, de hora a hora, pelo nó **Alert 1 every hour** (ver Figura 10), que desencadeia todo o fluxo descrito na secção 3.2 sem intervenção manual. No final do fluxo, cada oferta validada e transformada é publicada no *exchange* `amq.direct`, com a chave de encaminhamento `fila_vagas`.

A utilização de uma fila de mensagens desacopla a produção da persistência: o fluxo produtor limita-se a publicar as mensagens, enquanto o consumidor as processa ao seu próprio ritmo. Isto torna o sistema mais resiliente — se o PostgreSQL ficar indisponível, as mensagens aguardam na fila — e mais extensível, pois é possível acrescentar novos consumidores sem alterar o produtor.

A Figura 12 apresenta o painel de gestão do RabbitMQ (versão 3.13.7, sobre Erlang 26.2.5.16), a partir do qual foi monitorizado o comportamento do *broker*. Os indicadores globais observados encontram-se resumidos na Tabela 7.

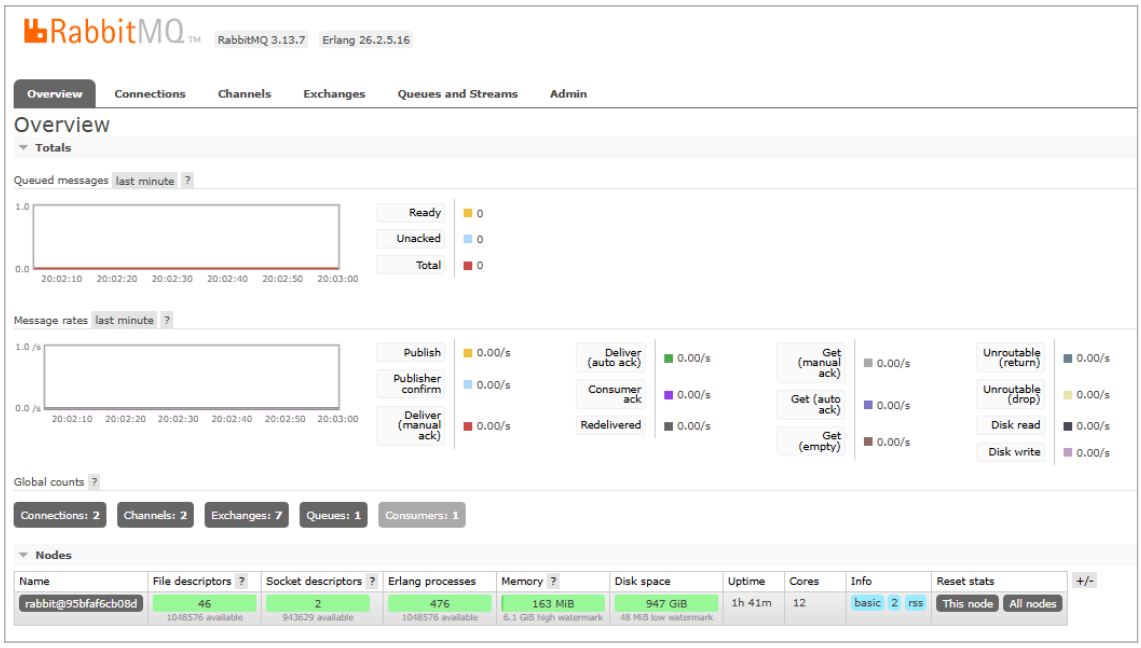


Figura 12: Painel de gestão do RabbitMQ (separador Overview).

Indicador	Valor	Interpretação
Connections	2	Ligações AMQP do produtor (publicação) e do consumidor (subscrição), ambos no Node-RED
Channels	2	Um canal por ligação
Exchanges	7	*Exchanges* predefinidos do *broker*, incluindo `amq.direct`
Queues	1	Fila `fila_vagas`
Consumers	1	Consumidor ativo (fluxo de carregamento no Node-RED)

Tabela 7: Indicadores globais observados no painel de gestão do RabbitMQ.

No instante da captura, a fila não apresenta mensagens prontas (*Ready*) nem por confirmar (*Unacked*), e as taxas de mensagens são nulas. Este comportamento é o esperado entre execuções horárias do *job*: o consumidor retira as mensagens da fila à medida que chegam, pelo que não se acumulam mensagens por processar.

4.3. Job de Carregamento e Escrita por Cima (Upsert)

O fluxo consumidor, representado na Figura 13, subscreve a fila através do nó *amq.direct: fila_vagas*, reconstrói os parâmetros no nó **Rebuild Params** (secção 3.2.3) e encaminha a mensagem em paralelo para dois destinos: o nó *debug 1*, que permite inspecionar os parâmetros reconstruídos, e o nó **TechPowerPostgres**, que executa a escrita na BD. O indicador *Queue: 0* apresentado por este último mostra que não existem pedidos pendentes de execução.

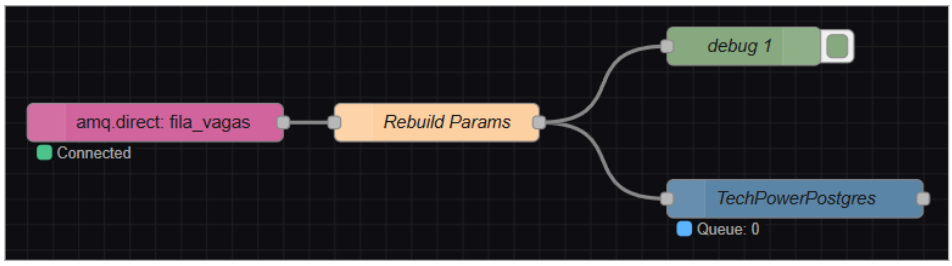


Figura 13: Fluxo consumidor e carregamento no PostgreSQL (Node-RED).

A escrita é feita através de uma instrução SQL parametrizada, em que os valores são passados por marcadores posicionais (`$1` a `$5`) preenchidos com o *array* `msg.params`. Para evitar a acumulação de registos repetidos a cada execução horária, a instrução recorre à cláusula `ON CONFLICT`, que converte a inserção numa escrita por cima (*upsert*): se a combinação de título e localização já existir, os restantes campos são atualizados em vez de ser criada uma nova linha.

```
INSERT INTO vagas_nodered (
    titulo_vaga, localizacao, salario_euros, nivel_experiencia, tipo_mercado
) VALUES ($1, $2, $3, $4, $5)
ON CONFLICT (titulo_vaga, localizacao)
DO UPDATE SET
    salario_euros     = EXCLUDED.salario_euros,
    nivel_experiencia = EXCLUDED.nivel_experiencia,
    tipo_mercado      = EXCLUDED.tipo_mercado;
```

A utilização de consultas parametrizadas, em vez da concatenação de texto, evita a injeção de SQL — relevante, uma vez que os títulos das vagas são texto livre proveniente de uma fonte externa — e garante o tratamento correto dos tipos de dados e dos carateres especiais.

4.4. Modelo de Dados no PostgreSQL

Ambas as vias persistem na mesma instância PostgreSQL, executada no contentor `tech_power_db`, mas em tabelas distintas. Esta separação é uma decisão de desenho deliberada: o nó **DB Writer** do KNIME opera em modo de substituição,

repondo a totalidade da tabela em cada execução do ciclo, o que apagaria os registos capturados em tempo real se ambas as vias partilhassem a mesma tabela.

4.4.1. Tabela ofertas_emprego (via em lote)

A tabela `ofertas_emprego` é escrita pelo KNIME e constitui a base analítica do *dashboard*. Armazena os onze atributos descritos na Tabela 8, que combinam os dados originais da oferta, os valores normalizados, os indicadores de enriquecimento e o indicador calculado. A ligação é estabelecida pelo nó **PostgreSQL Connector**, anotado com «Ligação à BD», que abre a sessão autenticada através do controlador *Java Database Connectivity* (JDBC) para a porta 5432 do contentor; o nó **DB Writer** executa em seguida a escrita em lote.

N.º	Coluna	Descrição
1	`Titulo_Vaga`	Título da oferta de emprego
2	`Localizacao`	Localização indicada na oferta
3	`Salario_Euros`	Salário anual normalizado e convertido para euros
4	`Nivel_Experiencia`	Nível de experiência (*Entry*, *Mid* ou *Senior*)
5	`Tipo_Mercado`	Classificação do mercado a que a oferta pertence
6	`Pais_Numbeo`	País normalizado, correspondente à entrada nos índices Numbeo
7	`Indice_Custo_Vida`	Índice de custo de vida do país (Numbeo)
8	`Indice_de_Renda`	Índice de renda (habitação) do país (Numbeo)
9	`Indice_Custo_Vida_Mais_Renda`	Índice combinado de custo de vida e renda (Numbeo)
10	`Taxa_Imposto`	Taxa de imposto aplicada ao salário
11	`Poupança Anual Estimada (€)`	Poupança líquida anual estimada, em euros

Tabela 8: Colunas da tabela ofertas_emprego, escrita pelo KNIME.

4.4.2. Tabela vagas_nodered (via contínua)

A tabela `vagas_nodered` foi criada especificamente para isolar a ingestão em tempo real, mantendo um esquema compacto de cinco colunas, alinhado com o *array* produzido pelo nó Normal Data.

```
CREATE TABLE vagas_nodered (
    titulo_vaga       VARCHAR(255),
    localizacao       VARCHAR(255),
    salario_euros     DOUBLE PRECISION,
    nivel_experiencia VARCHAR(255),
    tipo_mercado      VARCHAR(255)
);
```

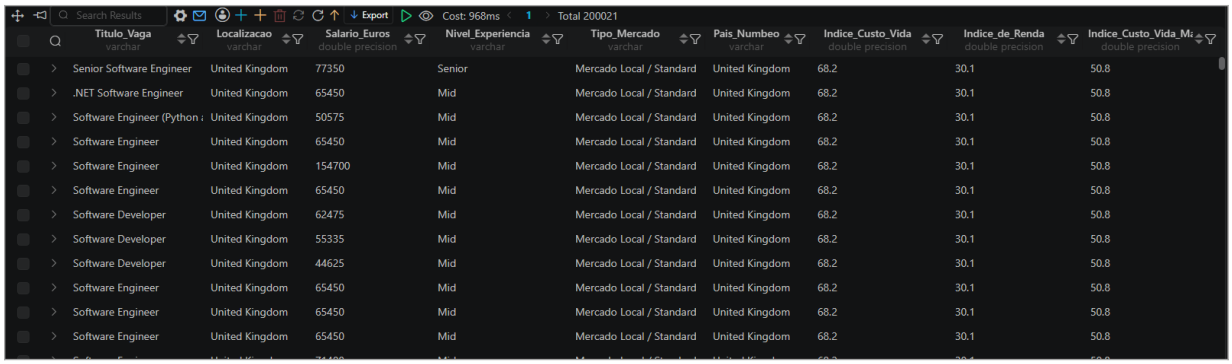
Note-se que, nesta tabela, a coluna `localizacao` armazena o país já normalizado pelo nó Normal Data, e não a localização original da oferta. Para viabilizar

a escrita por cima descrita na secção anterior, foi acrescentada uma restrição de unicidade sobre o par formado pelo título e pela localização, depois de esvaziada a tabela dos registos duplicados acumulados durante os testes iniciais.

```
TRUNCATE TABLE vagas_nodered;

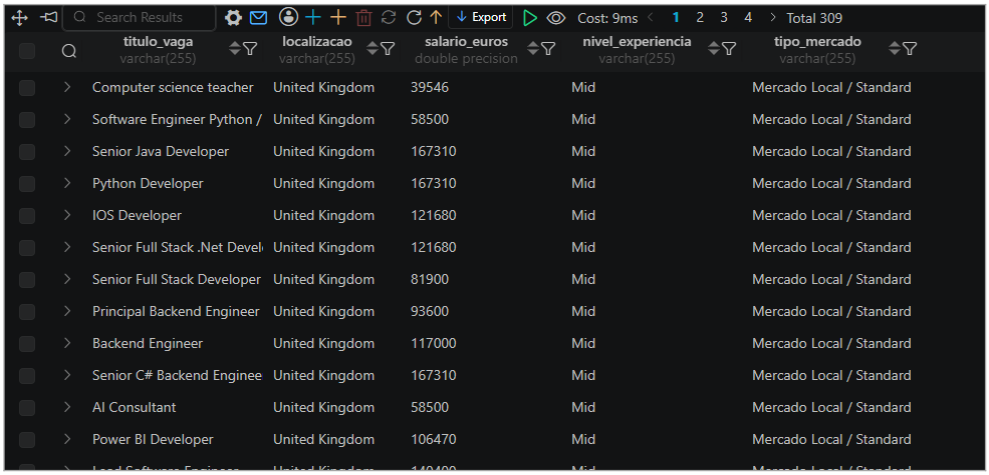
ALTER TABLE vagas_nodered
    ADD CONSTRAINT vaga_unica UNIQUE (titulo_vaga, localizacao);
```

As Figura 14 e Figura 15 apresentam amostras dos registos efetivamente persistidos em cada tabela, consultadas através de um cliente de BD. Confirmam a diferença de dimensão e de propósito entre as duas vias: a tabela `ofertas_emprego` acumula 200 021 registos, com as colunas económicas preenchidas, enquanto a tabela `vagas_nodered` contém 309 registos, resultado da deduplicação por *upsert* aplicada às ofertas recolhidas em tempo real.



Titulo_Vaga	Localizacao	Salario_Euros	Nivel_Experiencia	Tipo_Mercado	Pais_Numbeo	Indice_Custo_Vida	Indice_de_Renda	Indice_Custo_Vida_M…
Senior Software Engineer	United Kingdom	77350	Senior	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
.NET Software Engineer	United Kingdom	65450	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Engineer (Python …	United Kingdom	50575	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Engineer	United Kingdom	65450	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Engineer	United Kingdom	154700	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Engineer	United Kingdom	65450	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Developer	United Kingdom	62475	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Developer	United Kingdom	55335	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Developer	United Kingdom	44625	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Engineer	United Kingdom	65450	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Engineer	United Kingdom	65450	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8
Software Engineer	United Kingdom	65450	Mid	Mercado Local / Standard	United Kingdom	68.2	30.1	50.8

Figura 14: Registos da tabela ofertas_emprego, escrita pelo KNIME (200 021 registos).



titulo_vaga	localizacao	salario_euros	nivel_experiencia	tipo_mercado
Computer science teacher	United Kingdom	39546	Mid	Mercado Local / Standard
Software Engineer Python /	United Kingdom	58500	Mid	Mercado Local / Standard
Senior Java Developer	United Kingdom	167310	Mid	Mercado Local / Standard
Python Developer	United Kingdom	167310	Mid	Mercado Local / Standard
IOS Developer	United Kingdom	121680	Mid	Mercado Local / Standard
Senior Full Stack .Net Devel	United Kingdom	121680	Mid	Mercado Local / Standard
Senior Full Stack Developer	United Kingdom	81900	Mid	Mercado Local / Standard
Principal Backend Engineer	United Kingdom	93600	Mid	Mercado Local / Standard
Backend Engineer	United Kingdom	117000	Mid	Mercado Local / Standard
Senior C# Backend Enginee	United Kingdom	167310	Mid	Mercado Local / Standard
AI Consultant	United Kingdom	58500	Mid	Mercado Local / Standard
Power BI Developer	United Kingdom	106470	Mid	Mercado Local / Standard

Figura 15: Registos da tabela vagas_nodered, escrita pelo Node-RED (309 registos).

4.5. Job de Notificação: Alertas em Tempo Real via Telegram

A segunda saída do nó Normal Data alimenta o ramo de notificações, que concretiza o critério de mais-valia relativo ao envio automático de alertas previsto no

enunciado. O nó **Good Role Alert** avalia cada oferta à luz de dois critérios independentes, descritos na Tabela 9.

Critério	Condição	Justificação
Alto valor financeiro	Salário anual ≥ 100 000 €	Sinaliza ofertas no topo da escala salarial do mercado
Vaga incomum ou rara	Título contém uma palavra-chave de nicho	Sinaliza tecnologias de procura reduzida e elevada especialização, como *cobol*, *rust*, *blockchain*, *machine learning*, *quantum*, *fortran*, *elixir* ou *power electronics*

Tabela 9: Critérios de disparo de alertas no nó Good Role Alert.

A mensagem enviada é classificada dinamicamente em função dos critérios satisfeitos, distinguindo-se três situações: alto valor financeiro, vaga incomum ou rara e, quando ambos os critérios se verificam em simultâneo, a classificação combinada. O nó recorre ainda ao contexto do fluxo, através de `context.get` e `context.set`, para manter um contador incremental de alertas emitidos, que é anexado ao rodapé de cada mensagem juntamente com o cargo, o país, o nível, o tipo de mercado e o salário anual.

```
// Avaliação dos critérios de alerta
const titulo  = msg.payload[0].toLowerCase();
const salario = msg.payload[2];

const isValorAlto   = salario >= 100000;
const isRoleIncomum = PALAVRAS_CHAVE.some(k => titulo.includes(k));

if (!isValorAlto && !isRoleIncomum) { return null; }

let total = context.get("contadorVagas") || 0;
context.set("contadorVagas", ++total);
```

A Figura 16 apresenta quatro notificações recebidas no *bot*, todas classificadas como alerta de alto valor financeiro. Cada mensagem identifica o motivo do alerta e resume o cargo, o país, o nível, o tipo de mercado e o salário anual, permitindo ao destinatário avaliar a oferta sem consultar o *dashboard*.

O nó **http request** seguinte efetua um pedido HTTP POST ao *endpoint* `sendMessage` da API oficial do Telegram, enviando a mensagem formatada em *Markdown* para o *bot* `@TechPowerISIBot` (Telegram, s.d.). O identificador do *bot* e o *token* de autenticação são parâmetros do URL do pedido, pelo que são omitidos deste documento por razões de segurança.

```
POST https://api.telegram.org/bot<TOKEN>/sendMessage
```



Figura 16: Notificações recebidas no bot @TechPowerISIBot do Telegram.

4.6. Diagnóstico Operacional e Limitações

Durante o desenvolvimento foram identificadas e mitigadas duas situações operacionais relevantes:

- **Limitação de tráfego das APIs (HTTP 429).** Execuções manuais repetidas em curtos intervalos originam bloqueios temporários, com código de estado `429 Too Many Requests`, nas APIs da Reed e da Postcodes.io. As respostas inválidas daí resultantes são filtradas a montante, pelo filtro de qualidade do nó Normal Data e pela validação de estrutura do nó Rebuild Params, o que impede que cheguem erros à fila e à BD e mantém o indicador *Queue* do nó PostgreSQL a zero;

- **Sensibilidade a maiúsculas no PostgreSQL.** O motor normaliza para minúsculas os identificadores não delimitados por aspas. Os nomes das colunas da tabela `vagas_nodered` foram, por isso, declarados em minúsculas e sem aspas na instrução do nó TechPowerPostgres, ao contrário dos da tabela `ofertas_emprego`, criada pelo KNIME com nomes em maiúsculas e minúsculas, que exigem aspas duplas nas consultas do Grafana.

5. Visualização Analítica e Discussão de Resultados

Este capítulo apresenta a componente final da fase «L»: a visualização dinâmica dos dados persistidos, realizada no Grafana. Descrevem-se a ligação à BD e as variáveis de filtragem, os quatro painéis do *dashboard* e a discussão dos resultados obtidos.

5.1. Ligação e Variáveis Dinâmicas no Grafana

O Grafana, executado no contentor `tech_power_grafana`, foi ligado ao PostgreSQL através da fonte de dados `grafana-postgresql-datasource-1`, configurada para a BD do contentor `tech_power_db`. Sobre esta fonte de dados foram criadas duas variáveis globais do tipo *Query*, que alimentam listas de seleção no topo do *dashboard* e permitem filtrar todos os painéis em simultâneo. Ambas consultam a tabela `ofertas_emprego`, que é a base analítica enriquecida.

A variável **Pais** obtém os países disponíveis na tabela, por ordem alfabética, ignorando valores nulos:

```
SELECT DISTINCT "Pais_Numbeo"
FROM ofertas_emprego
WHERE "Pais_Numbeo" IS NOT NULL
ORDER BY "Pais_Numbeo" ASC;
```

A consulta devolve dez mercados: *Australia*, *Canada*, *France*, *Germany*, *India*, *Japan*, *Netherlands*, *Singapore*, *United Kingdom* e *United States*.

A variável **Nivel_Experiencia** obtém, do mesmo modo, os níveis de experiência existentes, devolvendo os três valores padronizados pelos nós Rule Engine do KNIME: *Entry*, *Mid* e *Senior*.

```
SELECT DISTINCT "Nivel_Experiencia"
FROM ofertas_emprego
WHERE "Nivel_Experiencia" IS NOT NULL
ORDER BY "Nivel_Experiencia" ASC;
```

5.2. Processo de Configuração de um Painel

A construção de cada painel seguiu sempre o mesmo processo, ilustrado na Figura 17 e na Figura 18 para o painel *Top 10 Países com Maior Poupança Anual*. Em primeiro lugar, seleciona-se o tipo de visualização no painel lateral direito — no caso, *Bar chart* com orientação horizontal — e define-se o título e a descrição que documenta o painel para quem o consulta.

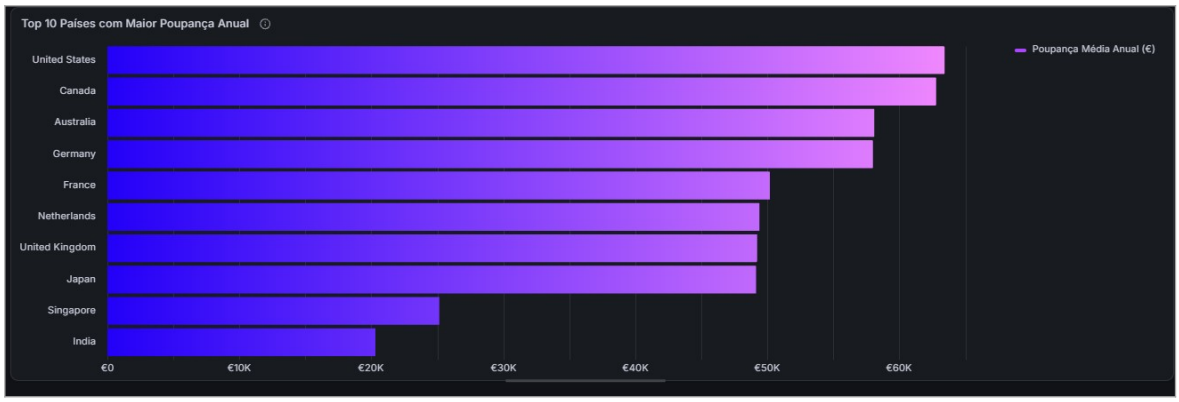


Figura 17: Configuração da visualização de um painel no Grafana.

Em segundo lugar, define-se a consulta no separador *Queries*, apresentado na Figura 18. Seleciona-se a fonte de dados `grafana-postgresql-datasource-1`, escolhe-se o formato de resposta `Table` e alterna-se o editor para o modo *Code*, que permite escrever SQL diretamente em vez de usar o construtor visual. O botão *Run query* permite validar o resultado antes de guardar.

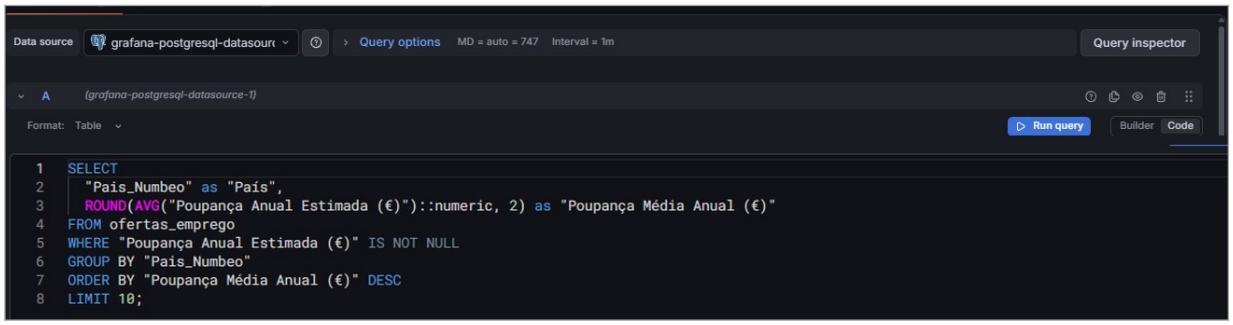


Figura 18: Editor de consultas SQL de um painel no Grafana.

Dois detalhes de implementação merecem nota. O primeiro é a necessidade de delimitar os nomes das colunas com aspas duplas, uma vez que a tabela `ofertas_emprego` foi criada pelo KNIME com nomes que combinam maiúsculas e minúsculas e incluem acentos e o símbolo do euro, como `"Poupança Anual Estimada (€)"`. O segundo é a conversão explícita de tipo na função de arredondamento, `ROUND(AVG(...)::numeric, 2)`, exigida pelo PostgreSQL porque a função `ROUND` com casas decimais não aceita o tipo `double precision` usado nas colunas numéricas.

5.3. Análise dos Painéis do Dashboard Tech Power Purchasing

O *dashboard Tech Power Purchasing*, apresentado na Figura 19, é composto por quatro painéis. Todos partilham a mesma cláusula de filtragem, que utiliza as variáveis descritas na secção anterior e faz com que as consultas se adaptem dinamicamente às seleções do utilizador:

```
WHERE "Pais_Numbeo" IN ($Pais)
  AND "Nivel_Experiencia" IN ($Nivel_Experiencia)
```



Figura 19: Dashboard Tech Power Purchasing no Grafana.

As consultas apresentadas nas subsecções seguintes correspondem à forma representativa de cada painel.

5.3.1. Top 10 Países com Maior Poupança Anual

Este gráfico de barras horizontais ordena os países pela poupança média anual estimada, resultante de uma agregação (AVG) agrupada por país. A consulta efetivamente utilizada, visível na Figura 18, é a seguinte:

```
SELECT
  "Pais_Numbeo" as "País",
  ROUND(AVG("Poupança Anual Estimada (€)")::numeric, 2) as "Poupança Média
Anual (€)"
FROM ofertas_emprego
WHERE "Poupança Anual Estimada (€)" IS NOT NULL
GROUP BY "Pais_Numbeo"
ORDER BY "Poupança Média Anual (€)" DESC
LIMIT 10;
```

Os Estados Unidos e o Canadá lideram, com valores acima dos 60 mil euros, seguidos pela Austrália e pela Alemanha. Seguem-se a França, os Países Baixos, o Reino Unido e o Japão, com valores próximos entre si, ficando Singapura e a Índia nas últimas posições. Este resultado evidencia o contributo do enriquecimento com os índices Numbeo: Singapura apresenta salários brutos elevados (ver painel seguinte), mas uma poupança estimada bastante inferior à dos restantes mercados de salário comparável, o que sugere que o custo de vida e da habitação absorve uma parte significativa do rendimento.

5.3.2. Poder de Compra – Média Salarial Anual

O gráfico circular representa o peso relativo da média salarial anual, em euros, de cada país:

```
SELECT "Pais_Numbeo" AS "País",
       AVG("Salario_Euros") AS "Salário Médio Anual (€)"
FROM ofertas_emprego
WHERE "Pais_Numbeo" IN ($Pais)
  AND "Nivel_Experiencia" IN ($Nivel_Experiencia)
GROUP BY "Pais_Numbeo";
```

Os Estados Unidos (14 %) e Singapura (13 %) apresentam as maiores fatias, seguidos da Austrália (12 %). Os restantes mercados situam-se entre 7 % e 11 %, com exceção da Índia, que ocupa a menor fatia. Este painel, lido em conjunto com o anterior, reforça a ideia central do trabalho: o salário bruto mais elevado não corresponde necessariamente à maior capacidade de poupança.

5.3.3. Salários por País e Senioridade

O gráfico de barras agrupadas compara, para cada mercado, quatro indicadores: o total de vagas, o salário mínimo, o salário médio e o salário máximo. A consulta combina funções de agregação (`COUNT`, `MIN`, `AVG` e `MAX`):

```
SELECT "Pais_Numbeo" AS "País",
       COUNT(*)              AS "Total de Vagas",
       MIN("Salario_Euros")  AS "Salário Mínimo (€)",
       AVG("Salario_Euros")  AS "Salário Médio (€)",
       MAX("Salario_Euros")  AS "Salário Máximo (€)"
FROM ofertas_emprego
WHERE "Pais_Numbeo" IN ($Pais)
  AND "Nivel_Experiencia" IN ($Nivel_Experiencia)
GROUP BY "Pais_Numbeo";
```

O salário médio e, sobretudo, o salário máximo são mais elevados nos Estados Unidos e em Singapura, seguidos da Austrália e do Canadá, enquanto a Índia se encontra entre os valores mais baixos. A amplitude entre o salário mínimo e o máximo é muito grande em todos os mercados, o que justifica a possibilidade de filtrar os resultados por nível de experiência através da variável *Nivel_Experiencia*.

5.3.4. Distribuição de Vagas por País

O painel de indicadores circulares (*gauges*) apresenta a contagem absoluta de ofertas por mercado, obtida com `COUNT(*)` agrupado por país. Os valores observados encontram-se na Tabela 10.

País	Vagas	Fração do total
United Kingdom	20 224	10,1 %
France	20 171	10,1 %
Canada	20 113	10,1 %
Japan	20 069	10,0 %
Germany	20 015	10,0 %
Netherlands	20 010	10,0 %
Singapore	19 962	10,0 %
United States	19 914	10,0 %
India	19 799	9,9 %
Australia	19 738	9,9 %
Total	**200 015**	**100 %**

Tabela 10: Número de vagas por mercado no painel Distribuição de Vagas por País.

O Reino Unido (20 224), a França (20 171) e o Canadá (20 113) apresentam o maior número de ofertas, mas as diferenças entre mercados são reduzidas: a Austrália, com o menor valor (19 738), tem menos de 2,5 % de ofertas do que o Reino Unido. A distribuição equilibrada, com cerca de 20 mil registos por mercado e um total próximo de 200 mil, decorre da dimensão do conjunto de dados histórico carregado pela via em lote e assegura que as comparações de salários e de poupança entre países não são enviesadas por diferenças relevantes no volume de amostras.

5.4. Síntese da Discussão e Limitações

Em conjunto, os painéis permitem responder à questão formulada na secção 1.2: o salário bruto mais elevado nem sempre se traduz em maior capacidade de poupança. Os Estados Unidos reúnem salários elevados e a maior poupança estimada, mas a comparação entre Singapura e países como o Canadá, a Austrália ou a Alemanha mostra que salários brutos semelhantes podem conduzir a poupanças bastante distintas, consoante os impostos e o custo de vida. A arquitetura híbrida acrescenta a esta leitura analítica uma camada de monitorização imediata: enquanto o *dashboard* responde à pergunta estrutural sobre onde compensa trabalhar, os alertas do Telegram sinalizam, à medida que surgem, as ofertas concretas que merecem atenção.

Importa, contudo, assinalar as limitações do estudo: (i) a taxa de câmbio utilizada na conversão para euros é fixa; (ii) a taxa de imposto é um valor estimado

único, pelo que não reflete escalões nem sistemas fiscais distintos; (iii) os índices do Numbeo são recolhidos no momento de cada execução, mas representam um retrato estático, sem série histórica; (iv) a anualização infere a periodicidade a partir da ordem de grandeza do valor, o que pode classificar incorretamente casos-limite; e (v) o gráfico circular representa pesos relativos de médias, pelo que deve ser lido como comparação de grandeza e não como partição de um total. Estas limitações são retomadas nos trabalhos futuros (secção 7.2).

6. Vídeo com Demonstração

Este capítulo disponibiliza o vídeo de demonstração do *pipeline* em funcionamento, exigido no ponto 6 do enunciado. O vídeo tem por objetivo validar, na prática, o funcionamento de toda a solução, evidenciando o arranque dos contentores no Docker Desktop, a execução do *workflow* no KNIME, o fluxo de extração e transformação no Node-RED, a publicação e o consumo de mensagens no RabbitMQ, o carregamento dos dados nas duas tabelas do PostgreSQL, a receção de um alerta no Telegram e a visualização no Grafana, incluindo a utilização dos filtros dinâmicos.

A ligação para o vídeo pode ser acedida através do código *Quick Response* (QR) da Figura 20 ou do endereço indicado abaixo, o qual consta igualmente do ficheiro `README.md` entregue com o trabalho.



Figura 20: Código QR com a ligação para o vídeo de demonstração do pipeline.

URL do vídeo: https://… (a preencher)

7. Conclusão e Trabalhos Futuros

7.1. Conclusão

O trabalho permitiu concretizar um *pipeline* de ETL completo para a análise do poder de compra no setor tecnológico, cumprindo as três fases exigidas no enunciado. Na fase «E», o *pipeline* consome dados em três formatos distintos — JSON de APIs REST, CSV de um ficheiro local e HTML extraído por XPath — a partir de quatro origens independentes. Na fase «T», aplica limpeza e validação, expressões regulares, geocodificação, conversão de estruturas, concatenação, junção relacional e cálculos financeiros. Na fase «L», persiste os resultados em duas tabelas relacionais, disponibiliza-os num *dashboard* dinâmico e despacha notificações automáticas.

Os objetivos definidos na secção 1.3 foram atingidos. A decisão arquitetural mais relevante foi a adoção de duas vias complementares: o KNIME assegura o processamento analítico pesado, com o cruzamento dos índices económicos e o cálculo da poupança real, enquanto o Node-RED atua como sonda de baixa latência, com alertas imediatos. A separação em duas tabelas revelou-se indispensável, uma vez que o modo de substituição do nó DB Writer apagaria os registos capturados em tempo real, e ilustra um compromisso real de integração de sistemas: duas ferramentas com cadências e modelos de escrita diferentes a partilhar o mesmo destino.

Do ponto de vista técnico, destacam-se a eficácia da limpeza de dados através dos filtros de qualidade, a resolução de localizações por geocodificação, a resiliência conferida pelo desacoplamento via RabbitMQ, a deduplicação assegurada pela escrita por cima e a capacidade analítica do PostgreSQL com o Grafana. A contentorização com Docker garantiu um ambiente isolado e reprodutível. Por último, o trabalho consolidou conceitos de integração de sistemas ao nível dos dados e a experiência na orquestração de múltiplas tecnologias e paradigmas.

7.2. Trabalhos Futuros

Como evolução do trabalho, identificam-se os seguintes desenvolvimentos:

- **Novas fontes de dados:** ligação a outras APIs de recrutamento, alargando a cobertura geográfica e a diversidade de ofertas;

- **Parâmetros dinâmicos:** atualização automática das taxas de câmbio a partir de uma API cambial, em substituição do valor fixo;
- **Fiscalidade mais realista:** substituição da taxa de imposto estimada única por escalões fiscais específicos de cada país;
- **Série histórica:** armazenamento dos índices do Numbeo ao longo do tempo, permitindo analisar a evolução do poder de compra em vez de um retrato instantâneo;
- **Senioridade automática:** extração do nível de experiência a partir do título da vaga na via contínua, com recurso a expressões regulares ou a técnicas de *Natural Language Processing* (NLP), substituindo o valor fixo atualmente atribuído;
- **Enriquecimento da via contínua:** cruzamento das novas ofertas com os índices já persistidos, de modo que os alertas do Telegram passem a indicar a poupança estimada e não apenas o salário bruto;
- **Orquestração centralizada:** migração do ciclo de repetição do KNIME para um agendador externo, como o Apache Airflow, com registo de execuções e tratamento automático de falhas.

8. Referências Bibliográficas

Docker. (s.d.). *Docker Documentation*. Obtido de Docker Docs: https://docs.docker.com/

Ferreira, L., & Ribeiro, Ó. (2026). *Integração de Sistemas de Informação – Trabalho Prático I (enunciado)*. Escola Superior de Tecnologia, Universidade Politécnica do Cávado e do Ave.

Grafana Labs. (s.d.). *Grafana documentation*. Obtido de Grafana Labs: https://grafana.com/docs/grafana/latest/

Kaggle. (s.d.). *Latest Data Science Salaries*. Obtido de Kaggle: https://www.kaggle.com/datasets

KNIME. (s.d.). *KNIME Analytics Platform documentation*. Obtido de KNIME: https://docs.knime.com/

MDN Web Docs. (s.d.). *Regular expressions*. Obtido de MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions

MongoDB. (s.d.). *MongoDB Documentation*. Obtido de MongoDB: https://www.mongodb.com/docs/

Node-RED. (s.d.). *Node-RED: Low-code programming for event-driven applications*. Obtido de Node-RED: https://nodered.org/

Numbeo. (s.d.). *Cost of Living Index*. Obtido de Numbeo: https://www.numbeo.com/cost-of-living/

Postcodes.io. (s.d.). *Postcodes.io API documentation*. Obtido de Postcodes.io: https://postcodes.io/docs

PostgreSQL Global Development Group. (s.d.). *PostgreSQL 16 Documentation*. Obtido de PostgreSQL: https://www.postgresql.org/docs/16/

RabbitMQ. (s.d.). *RabbitMQ documentation*. Obtido de RabbitMQ: https://www.rabbitmq.com/docs

Reed. (s.d.). *Reed.co.uk Jobseeker API*. Obtido de Reed: https://www.reed.co.uk/developers/jobseeker

Telegram. (s.d.). *Telegram Bot API*. Obtido de Telegram: https://core.telegram.org/bots/api